NEXORA Blockchain Plan *(Plain English)*

A simple guide to how we add blockchain and smart contracts to NEXORA. No jargon walls — read top to bottom.

---

1. What are we actually building?

NEXORA is a prediction market — people bet on "will X happen?" with real money. Right now, everything is fake: mock prices, fake balances, data lost on server restart.

We want to make it real. That means:

- Users hold their own money (no "trust us, we have your funds").
- Trades and payouts happen automatically by code, not by us pressing buttons.
- Nobody — including us — can secretly change balances or reverse trades.

Blockchain gives us those properties for free. Smart contracts are how we write the rules.

2. What is a smart contract? (Easy version)

Think of a **vending machine**.

- You put money in.
- You press a button.
- It gives you a snack.

Nobody has to trust the vending machine operator. The machine just follows its rules. If the rules are written correctly, nobody can cheat.

A smart contract is the same thing, but for money and digital assets. We write a program, put it on the blockchain, and from then on:

- Users send money (USDC) into it.
- It follows the rules we wrote.
- When a market resolves, it automatically pays the winners.

We can't "turn it off" or "refund someone." Whatever we code is what happens. This is powerful — and dangerous if we mess up. That's why we test a lot and get an audit before real money.

3. What is a "platform"?

A **blockchain platform** is the computer network the contract runs on. Different platforms have different trade-offs: some are cheap, some are fast, some are very secure, some use a different programming language.

For this project, the choice narrows down quickly.

The short version

- **Ethereum** — most trusted, but every trade costs $1–$20. Too expensive for a betting site.
- **Polygon** — runs Ethereum-style contracts, but trades cost less than a cent. This is what Polymarket itself uses.
- **Base, Arbitrum, Optimism** — also cheap, also Ethereum-style. Fine backups.
- **Solana, Sui, Aptos** — cheap and fast, but use a different programming language. We'd have to rewrite everything from scratch.

Our pick: Polygon

Three reasons:

1. **Cheapest user experience.** A trade costs pennies, not dollars.
2. **Same platform as real Polymarket.** We can reuse their tools and audited code.
3. **Ethereum-compatible.** The code we write works on Base / Arbitrum / Optimism too if we ever want to move.

We start on **Polygon Amoy** (a free "testnet" for practice) and only move to real money once it's safe.

4. What language will we write contracts in?

Solidity. It's the most popular language for smart contracts and the only one that matters for Ethereum-style chains. Looks a bit like JavaScript. ~500–800 lines of new Solidity for this whole project — most of the hard parts are already written by other teams.

5. What do we build vs. reuse?

Writing smart contracts from scratch is how people lose millions of dollars. So we reuse as much audited code as possible.

Stuff we reuse (don't build)

- **USDC (the money).** Circle already made it. We just point to their address.
- **Conditional Tokens** — the system that turns a bet into "YES shares" and "NO shares." Built by a company called Gnosis. Already deployed on Polygon. We just use it.
- **OpenZeppelin** — a library of safe building blocks (access control, pause switches, etc.).

Stuff we build (our code)

1. **Market AMM (Automated Market Maker)** — a little vending machine per market. Holds YES and NO shares, quotes prices, takes a small fee.
2. **Factory** — a contract that creates a new AMM whenever an admin makes a new market.
3. **Resolver** — the contract an admin calls to declare a winner. Later, this can be replaced by an oracle (more on that below).

That's it. Three contracts. Everything else is reused.

6. How resolution works (the "who won?" problem)

The hardest part of a prediction market is deciding who won. If we bet on "Will team X win?" something has to officially say yes or no.

Two options:

Option A: Admin decides (simple)

An admin checks the result, logs in, and marks the market resolved. Fast, cheap, but users have to trust the admin.

Option B: Oracle decides (decentralized)

A system called **UMA Optimistic Oracle** (what real Polymarket uses) lets anyone propose an answer, and anyone can challenge it within a time window. If nobody challenges, the answer sticks. Slower, but trustless.

Our plan: start with Option A (admin), add Option B later. Using a **multisig wallet** — meaning 3 out of 5 trusted people must agree — so one admin can't rug anyone.

7. The full journey (step by step)

Step 1 — Set up the contract project (1 week)

- Install **Foundry** (a popular tool for writing and testing Solidity).
- Write the three contracts above.
- Write tests for every scenario (make a market, buy YES, sell, resolve, pay winners).
- Deploy to Polygon Amoy (testnet, free).

Step 2 — Connect wallet to the website (3 days)

- Add **RainbowKit** to the site — the pretty "Connect Wallet" button.
- Users connect MetaMask (or similar).
- Show their real USDC balance in the navbar.
- Replace mock trades with real blockchain transactions.

Step 3 — Mirror the blockchain into our database (3 days)

- Reading straight from the blockchain is slow. So we run an **indexer** — a small program that watches the chain and copies everything into our normal database.
- That way the website stays fast, while the blockchain stays the source of truth.
- We'll use a tool called **Ponder** or **Envio** for this (built exactly for the job).

Step 4 — Resolution flow (2 days)

- Admin opens the admin page, picks the winning outcome.

- The page helps them sign a multisig transaction.
- Once 3 of 5 admins sign, the contract pays out.
- Winners see a "Claim" button and collect their USDC.

Step 5 — Safety checks (1 week minimum)

- Run automated tools (**Slither**, **Mythril**) that look for common bugs.
- Fuzz tests — throw random inputs at contracts and see what breaks.
- Put a "pause" switch on everything in case we spot an issue.

Step 6 — Before real money

- **Professional audit** by a security firm. Costs $10k–$30k. **Non-negotiable** if real funds are involved.
- **Bug bounty** — pay hackers to tell us about holes before black hats find them.
- Only then flip to Polygon mainnet.

8. Timeline summary

Phase	Time	What ships
Testnet demo	3–4 weeks	Fully working prediction market on Polygon Amoy with fake money
Audit + fixes	4–6 weeks	Professional review, all issues fixed
Mainnet launch	1 week	Real USDC, real money, real users

Total: ~2–3 months to real mainnet launch.

9. Costs

Thing	Cost
Testnet development	$0 (free fake tokens from a faucet)
Deploying to mainnet	~$5 in gas fees
Each user trade	$0.001–$0.01 in gas (user pays)
Professional audit	$10k–$30k
Optional bug bounty	$1k–$10k per issue found

If we skip the audit, we stay on testnet and call it a demo. Putting unaudited contracts on mainnet with real money = how the news stories start.

10. Risks and how we handle them

Risk	Our plan
Bug in our contract loses funds	Audit, tests, pause switch, start small
Admin goes rogue	Multisig (3-of-5), public transparency of every action
User loses wallet	We can't help — they're non-custodial. We show a clear warning at signup
Regulation changes	We're non-custodial and may need to geo-block certain countries
Polygon has problems	Contracts are portable — we can redeploy on Base or Arbitrum

11. What changes on the website?

Old (mock)	New (blockchain)
Fake balance in `WalletContext`	Real USDC balance read from chain
Click "Trade" → server updates a number	Click "Trade" → wallet popup → user signs → blockchain confirms
Admin resolves by clicking a button	Admin resolves via multisig → contract pays automatically
Email/password login	"Sign in with wallet" (Sign-In With Ethereum)
Data lost on restart	Data permanent, on-chain

Most of the UI stays the same. The plumbing underneath changes.

12. The next concrete step

If this all sounds right, the first thing to do is:

> Set up the `contracts/` folder with Foundry and write the factory + AMM + resolver with tests. Deploy to Polygon Amoy. End-to-end happy path: create a market → buy shares → resolve → winners claim.

Once that works on testnet, everything else (website integration, indexer, admin flow) is ordinary work.

Last updated: 2026-04-21